



MedX

Developer & Admin Guide

Architecture, APIs, cryptography, deployment and day-to-day vendor operations for the MedX platform — the desktop LIMS, the admin panel, and the patient website.

Document	Audience	Version	Date
Developer & Admin Guide	Platform owner (vendor) & developers	1.0 (App v0.1.0)	15 July 2026

MedX — Offline-first Pathology Lab Management for Indian Labs

Contents

1. System Overview & Architecture
2. Repository Layout & Tech Stack
3. Desktop App (apps/medx-app) Internals
4. Admin Panel (apps/medx-admin) Internals
5. Patient Website (apps/medx-web) Internals
6. Licensing Cryptography & Key Rotation
7. Cloud Storage Reference (Vercel Blob)
8. API Reference
9. Environment Variables & Secrets
10. Deployment Runbook
11. Admin Panel Operations Guide (Vendor Handbook)
12. Business Rules Summary
13. Security Model & Hardening History
14. Admin Troubleshooting

1. System Overview & Architecture

MedX consists of three connected platforms:

Platform	Runs on	Purpose
medx-app — Desktop LIMS	Lab PCs (Windows, Electron)	The product labs buy. Offline-first; all lab data lives locally. Talks to the cloud only for license heartbeats, portal sync (Pro+), and online-booking pulls.
medx-admin — Admin panel	Vercel (Next.js 16)	The vendor's control tower: issue/renew/revoke licenses, device limits, CRM, payments ledger, helpdesk, remote feature config, platform status monitoring.
medx-web — Patient website	Vercel (Next.js 16)	Public site: report portal (invoice+phone), home-collection booking, lab directory, test pricing. Receives sync pushes from desktop apps.

Data flow

- **Desktop** → **Admin**: heartbeat POST every 5 min + at startup/activation. Carries licenseKey, deviceId, hostname, and the current token. Response delivers revocation, renewed tokens, and admin messages.
- **Desktop** → **Web**: full sync push (settings, catalog, orders, patients) every 5 min + debounced on change; authenticated by the signed license token. Pro/Enterprise only.
- **Web** → **Desktop**: online bookings stored per lab; desktop GETs them (Bearer license token), imports, then ACKs so the cloud copy is deleted.
- **Web** → **Admin**: the public labs-directory endpoint validates sync/booking targets and feeds the site's lab list.
- **Admin** ↔ **Blob**: all cloud state persists in one private Vercel Blob store shared by admin and web.

2. Repository Layout & Tech Stack

```
MEDX-SOFTWARE/  
apps/  
medx-app/ Electron + React 19 + Vite + Zustand (desktop LIMS)  
electron/ main.ts, preload.ts, db/ (SQLite w/ JSON fallback)  
src/core/ gst, numbering, ranges, licensing, sync, windowSync,  
interfacing (ASTM/HL7), pdfReport  
src/data/ store.ts (Zustand), types.ts  
src/features/ 30+ page components  
scripts/ dev/build-electron.mjs, generate-keys.mjs  
medx-admin/ Next.js 16 admin panel (App Router, src/)  
src/lib/ adminDb, cloudStore, adminAuth, licenseVerify  
src/app/api/ licenses, heartbeat, labs-directory, status, auth,  
crm, billing, tickets, config  
src/proxy.ts auth gate (Next 16 middleware)  
medx-web/ Next.js 16 patient site  
src/lib/ cloudStore, licenseVerify  
src/app/api/ sync, bookings, health  
docs/ ARCHITECTURE, FEATURES-AND-PLANS, ROADMAP, manuals/  
Database/ local-dev JSON fallback storage
```

Layer	Technology
Desktop shell	Electron (main + preload, contextIsolation, IPC allow-list)
Desktop UI	React 19, TypeScript, Vite, Zustand (persisted), HashRouter
Desktop storage	better-sqlite3 via IPC; JSON-file fallback; localStorage in browser dev
Web apps	Next.js 16.2 (App Router), React 19, TypeScript
Cloud storage	Vercel Blob (private store 'medx-db'), @vercel/blob SDK
Crypto	ECDSA P-256 (WebCrypto) license tokens; HMAC-SHA256 admin sessions
Packaging	electron-builder (NSIS installer + portable), electron-updater (GitHub releases)
Tests	Vitest — 19 unit tests on money/GST/numbering/ranges

3. Desktop App (apps/medx-app) Internals

Processes & persistence

- The renderer never touches Node — everything crosses the preload bridge (window.medx.*): store get/set, hostname, TCP simulate, serial ports, updater, cross-window broadcast.
- Zustand store 'medx-store-v1' persists through the bridge to the main process (SQLite/JSON at the OS user-data dir); plain localStorage in browser dev.
- The counter display window (#/display) is **read-only**: its storage adapter no-ops writes and background engines don't start there (a stale-snapshot write would clobber the main window's data).
- Cross-window queue sync: token changes broadcast via IPC relay in main.ts (BroadcastChannel fallback in dev). See src/core/windowSync.ts.

Background engines (src/core/sync.ts)

- startSyncEngine() runs only in the main window: every 5 min → forceCloudSync() + pullWebBookings() + runHeartbeat(); plus a 1.5 s debounced sync on order/patient changes and an initial run ~3 s after launch.
- forceCloudSync() no-ops unless an active non-Starter license and token exist.
- pullWebBookings() GETs /api/bookings with Bearer token, imports new visits (deduped by webRef), then PATCHes ackIds.
- runHeartbeat() → checkLicenseHeartbeat(): handles onRevoked (drop license), onTokenRefresh (verify signature locally, then store), onMessage (adminNotice banner).

Licensing client (src/core/licensing.ts)

- verifyLicenseToken(): splits base64url(payload).base64url(sig), verifies ECDSA P-256 against the embedded public key. Activation and boot both re-verify.
- Route gating: isRouteAllowed(path, tier) — Enterprise routes (/qc, /tpa, /abdm, ...) and Pro routes (/doctors, /interfacing, ...) lock with an in-app upsell screen.
- Device identity: random persistent deviceId in localStorage + OS hostname via IPC.
- Heartbeat 'unknown_key' responses never deactivate — only 'revoked' and 'device_limit' do (offline-first: a validly signed token survives server hiccups).

Analyzer interfacing

- electron/main.ts hosts a TCP server on port 8100; complete frames are forwarded to the renderer over IPC.
- src/core/interfacing.ts parses ASTM E1394 (O/R records) and HL7 (OBR/OBX), maps analyte codes, and the store auto-writes matched results + audit entries.
- Serial (RS-232) support and a loopback simulator live on the Machine Interfacing page.

4. Admin Panel (apps/medx-admin) Internals

Authentication

- src/proxy.ts (Next 16's middleware) fails closed on every route except /login, /api/auth, /api/heartbeat, /api/labs-directory.
- Login: POST /api/auth compares against ADMIN_PASSWORD (constant-time via SHA-256 digests) and sets an httpOnly cookie holding `exp.HMAC(exp)` signed with SESSION_SECRET (7-day TTL).
- Automation/scripts can instead send `Authorization: Bearer <ADMIN\_PASSWORD>` to any protected API.

Storage layer

- src/lib/cloudStore.ts: readJson/writeJson/listJson/deleteJson over the private Vercel Blob store; local Database/*.json fallback when BLOB_READ_WRITE_TOKEN is absent (dev).
- readJson throws on unexpected storage errors (returns the fallback only for missing keys) so read-modify-write flows can't clobber data with defaults.
- **Licenses are one document each** (license/<id>) — the original single admin-keys array suffered read-modify-write races between concurrent serverless invocations that silently erased records. getLicenses() migrates any legacy blob once, then drains it.

Heartbeat control channel (api/heartbeat)

- Request: { licenseKey, deviceId, hostname, currentToken? }.
- Registers/updates the device (limit per tier), stamps lastHeartbeatAt.
- Returns { active, tier, validUntil, code?, token?, adminMessage? }.
- The re-signed token is returned **only** when currentToken's signature verifies for the same licenseKey (expiry deliberately ignored so renewals reach expired installs). A bare licenseKey — which is public via the directory — can never extract a token.

Status aggregation (api/status + /status page)

Probes the blob store (write+read round-trip) and medx-web /api/health, then joins licenses with synced lab docs and pending booking counts to render per-lab connectivity (heartbeat/sync freshness, device counts, data volumes).

5. Patient Website (apps/medx-web) Internals

- api/sync (POST): requires licenseToken in the body; verifies signature server-side (lib/licenseVerify.ts), rejects unknown/revoked keys and Starter tier; writes labs/<licenseKey> document.

- `api/bookings`: POST is public (patients) but only for active Pro/Enterprise labs; GET/PATCH require Bearer license token (proof of lab identity); ACKed bookings are deleted (minimal PHI retention, cap 200 pending).
- `api/health` (GET): blob connectivity + latency for the admin status page.
- `portal/actions.ts` (server actions): `verifyAndFetchReport` matches invoice across per-lab docs then enforces last-10-digit phone match; `getSyncedLabs` merges the admin directory with synced docs (revoked labs filtered).
- The UI is a mobile-first PWA-style site (manifest + add-to-home-screen) with a 2026-refresh design system in `globals.css` (aurora hero, glass cards, bento grid).

6. Licensing Cryptography & Key Rotation

- Token format: `base64url(JSON payload) + '.' + base64url(ECDSA-P256-SHA256 signature over the base64url payload string)`.
- Payload fields: `licenseKey`, `labName`, `contactPhone`, `issuedAt`, `validUntil`, `tier`.
- The **private key** exists only in the `LICENSE_PRIVATE_KEY_JWK` env var on `medx-admin`. The **public key** is embedded in `medx-app` (`licensing.ts`), `medx-web` and `medx-admin` (`licenseVerify.ts`).
- Renewals/tier changes re-sign the token server-side; clients verify locally before accepting (a compromised transport cannot forge licenses).

Rotating the keypair

- Generate: `node apps/medx-app/scripts/generate-keys.mjs`
- Set the new private JWK in Vercel env `LICENSE_PRIVATE_KEY_JWK` (all environments) and redeploy `medx-admin`.
- Replace `PUBLIC_KEY_JWK` in `medx-app/src/core/licensing.ts`, `medx-web/src/lib/licenseVerify.ts`, `medx-admin/src/lib/licenseVerify.ts`.
- Rebuild/redeploy all three; reissue any outstanding license tokens (old ones become invalid).
- History: rotated 2026-07-15 after the original private key was committed to the public repo.

7. Cloud Storage Reference (Vercel Blob)

Store: **medx-db** (private, region `iad1`), shared by `medx-admin` and `medx-web` via `BLOB_READ_WRITE_TOKEN`.

Document key	Contents	Written by
<code>license/<LIC-ID></code>	One license record: token, lab info, tier, validity, status, <code>devices[]</code> , <code>lastHeartbeatAt</code> , <code>adminMessage</code>	<code>medx-admin</code> (<code>issue/lifecycle/heartbeat</code>)
<code>admin-keys</code>	LEGACY license array — drained by one-time migration; kept empty	<code>medx-admin</code> (migration only)
<code>labs/<LIC-ID></code>	Lab's portal partition: settings, catalog, orders, patients, <code>lastSyncedAt</code>	<code>medx-web /api/sync</code>
<code>bookings/<LIC-ID></code>	Pending online home-collection bookings (deleted after desktop ACK)	<code>medx-web /api/bookings</code>

Document key	Contents	Written by
admin-customers	CRM pipeline records with notes	medx-admin
admin-payments	Vendor revenue ledger (paise)	medx-admin
admin-tickets	Helpdesk tickets with message threads	medx-admin
admin-config	Global feature gates (ABDM/WhatsApp/SMS/maintenance)	medx-admin
admin-health-probe / health-probe	Status-page connectivity probes	status/health endpoints

8. API Reference

medx-admin (<https://medx-admin-lac.vercel.app>)

Endpoint	Auth	Description
POST /api/auth	public	{password} → session cookie. DELETE = logout.
GET /api/licenses	admin	Full license records incl. tokens & devices.
POST /api/licenses	admin	{labName, contactPhone, tier, validDays} → signed license.
PATCH /api/licenses	admin	{id, action} where action ∈ revoke reactivate extend{days} setTier{tier} removeDevice{deviceId} setMessage{message} delete (revoked only). No action = revoke.
POST /api/heartbeat	public	{licenseKey, deviceId, hostname, currentToken?} → {active, tier, validUntil, code?, token?, adminMessage?}.
GET /api/labs-directory	public	Sanitized: {id, labName, contactPhone, tier, status}. No tokens/devices.
GET /api/status	admin	Platform checks + per-lab connectivity summary.
GET/POST/PATCH /api/crm /api/billing /api/tickets /api/config	admin	CRM customers/notes/stages, payments ledger, helpdesk tickets, global feature gates.

medx-web (<https://medx-web-one.vercel.app>)

Endpoint	Auth	Description
POST /api/sync	license token in body	Full lab push → labs/<id>. 401 invalid token; 403 inactive or Starter.
POST /api/bookings	public	Patient booking {labId, name, phone, address, date, timeSlot, age?, tests?}. 403 for inactive/Starter labs.
GET /api/bookings	Bearer license token	Pending bookings for the token's lab.
PATCH /api/bookings	Bearer license token	{ackIds:[...]} deletes delivered bookings.
GET /api/health	public	{ok, blob, latencyMs, time}.

9. Environment Variables & Secrets

Variable	Project	Purpose
BLOB_READ_WRITE_TOKEN	admin + web	Private blob store access (auto-managed by Vercel for admin; mirrored to web).
ADMIN_PASSWORD	admin	Panel login + Bearer automation auth.
SESSION_SECRET	admin	HMAC key for session cookies.
LICENSE_PRIVATE_KEY_JWK	admin	ECDSA P-256 private JWK (JSON) — the crown jewels.
MEDX_ADMIN_URL	web	Admin base URL for directory validation (defaults to production).
MEDX_WEB_URL	admin (optional)	Web base URL for status probe (defaults to production).

Important: Secrets exist only in Vercel env vars and git-ignored `.env.local` files. Never commit them; the repo is public. Local dev values can be re-pulled with `vercel env pull .env.local --yes` in each app.

10. Deployment Runbook

Web deployments (Vercel team: medx-lab)

```
cd apps/medx-admin && npm run build && npx vercel --prod --yes
cd apps/medx-web && npm run build && npx vercel --prod --yes
```

- Production aliases: `medx-admin-lac.vercel.app` and `medx-web-one.vercel.app`.
- Env var changes require a redeploy to take effect.
- Blob store management: `npx vercel blob list-stores` (store: `medx-db`).

Desktop build & release

```
cd apps/medx-app
npx vitest run # 19 unit tests
npx tsc --noEmit # typecheck
npm run build # vite renderer + electron bundle
npm run package # NSIS installer + portable exe -> release/
```

- Artifacts: `release/MedX-Setup-0.1.0.exe` (installer) and the portable exe.
- `electron-updater` is configured to pull updates from GitHub releases — publish the artifacts to a release tagged with the version to enable in-app auto-update.
- Reinstalls preserve the user's data and license (Electron `user-data` dir is untouched).

Local development

```
apps/medx-app: npm run dev # vite on :5173 (browser, localStorage)
npm run dev:electron # full Electron shell
apps/medx-admin: npm run dev # :3000 (needs .env.local)
apps/medx-web: npm run dev # :3000 (needs .env.local)
```

11. Admin Panel Operations Guide (Vendor Handbook)

Selling a license

- Deployed Labs → Generate License Key → lab name, phone, tier, validity days → copy the token and send it to the lab (WhatsApp/email).
- The lab pastes it in Settings → System License → Activate. Their device appears in the panel within seconds of activation (watch Devices / Last heartbeat).

Renewals & upgrades (lab pays again)

- Deployed Labs → Manage → **+30d/+90d/+365d or custom Extend**. The token is re-signed; the lab's app applies the new expiry automatically on its next check-in (≤5 min while the app is open, or instantly via their "Check License Status Now" button).
- Change Plan works the same way — modules unlock/lock remotely with no reinstall.
- Record the payment in the CRM's billing ledger to keep the revenue dashboard accurate.

Enforcement & device management

- **Revoke** drops the lab to Starter at its next check-in (data untouched). **Reactivate** restores it.
- **Remove device** frees a slot when a lab replaces a PC; the new PC registers on its next heartbeat.
- **Delete Permanently** is only allowed on revoked licenses (guardrail against wiping paying customers).

Messaging & monitoring

- Manage → Message to Lab: the text appears as a banner inside that lab's app on its next check-in; Clear removes it.
- Platform Status: green/amber/red freshness for every lab's heartbeat and portal sync, device counts, synced data volumes, pending bookings, plus health of the blob store and patient site (auto-refreshes every 30 s).

12. Business Rules Summary

Rule	Value
Plans	Starter (one-time) · Pro · Enterprise (see FEATURES-AND-PLANS.md for pricing)
Device limits	Starter 1 · Pro 3 · Enterprise unlimited (999)
Starter daily cap	15 patients/day, WhatsApp delivery locked
Portal & online bookings	Pro and Enterprise only; Starter labs are fully offline (privacy guarantee)
Heartbeat cadence	App start, license activation, then every 5 minutes; manual button in Settings
Sync cadence	Every 5 min + 1.5 s debounce after order/patient changes
Revocation/renewal latency	Next check-in (≤5 min with app open; instant via manual check)
Data ownership	All lab/patient data lives on the lab's PC; cloud holds licensing metadata + Pro+ portal copies

13. Security Model & Hardening History

- **Admin panel:** fail-closed proxy auth on every page/API; public surface is only heartbeat + sanitized directory + login.

- **License forgery:** impossible without the private key (env-only). Clients verify every token locally, including heartbeat-delivered refreshes.
- **Sync/bookings impersonation:** blocked — lab identity requires the signed token, not the public lab ID.
- **Token exfiltration via heartbeat:** blocked — refresh requires proof-of-possession of a genuinely signed token.
- **PHI at rest:** private blob store (token-authenticated); bookings deleted after desktop ACK; Starter data never leaves the PC.
- **Race safety:** per-license documents; storage reads throw (not default) on transient failures.

Hardening changelog (2026-07-15)

- Replaced dead kvdb.io buckets (all cloud writes were silently lost) with the private Vercel Blob store.
- Added panel auth — the panel was previously public, and /api/licenses leaked full tokens.
- Rotated the ECDSA keypair after the private key was found committed to the public repo.
- Moved license storage to per-document records after write races erased a live license.
- Made the desktop keep validly-signed licenses on 'unknown_key' heartbeat responses.

14. Admin Troubleshooting

Symptom	Diagnosis & fix
Lab shows Devices 0 / heartbeat 'never'	Their app hasn't checked in: confirm they run the latest build, are online, and the license is Active. Ask them to press "Check License Status Now". Verify server-side with a curl heartbeat.
Revoke didn't stop the app	Expected until the lab's next check-in (≤5 min online). If they stay offline indefinitely they retain features — by design for offline-first; follow up directly.
Renewal not visible at the lab	The app needs one online check-in. The refreshed token only flows if their stored token's signature is valid — pre-rotation tokens require issuing a fresh key.
Portal empty for a lab	Check Platform Status → last portal sync. Starter labs never sync. Confirm their app shows the green sync dot.
Status page shows blob store DOWN	Check Vercel Blob status / BLOB_READ_WRITE_TOKEN validity; verify with vercel blob list-stores.
Locked out of the panel	Reset ADMIN_PASSWORD in Vercel env vars and redeploy medx-admin.
Booking POST returns 403	Target lab is Starter, revoked, or expired in the directory — exactly as designed.

End of guide. Companion documents: docs/ARCHITECTURE.md, docs/FEATURES-AND-PLANS.md, docs/ROADMAP.md, and the MedX User Manual PDF for lab-facing features.